

Índice

2	Spikes de materia oscura alrededor de agujeros negros supermasivos: implementación computacional y documentación del código	2
2.1	Objetivo y alcance	2
2.2	Organización de la librería	2
2.2.1	Responsabilidades de los módulos	3
2.3	Dependencias	3
2.4	Convenciones físicas y unidades	4
2.5	Módulo <code>density.py</code> : integración	4
2.5.1	Modelo de Gondolo-Silk	4
2.5.2	Eliminación de la singularidad en el extremo angular	5
2.5.3	Estabilidad numérica	5
2.5.4	Referencia de funciones	6
2.6	Módulo <code>isothermal.py</code> : núcleo finito	7
2.6.1	Modelo implementado	7
2.6.2	Referencia de funciones	7
2.7	Módulo <code>adiabatic_cusp.py</code> : cúspide adiabática	8
2.7.1	Perfil inicial y función de distribución	8
2.7.2	Evaluación logarítmica de la función de distribución	8
2.7.3	Acciones radiales y mapeo adiabático	9
2.7.4	Solución en la variable $\log E$	9
2.7.5	Referencia de funciones auxiliares	10
2.8	Módulo <code>storage.py</code>	12
2.9	Módulo <code>__init__.py</code>	13
2.10	Flujos de cálculo	14
2.10.1	Caso isotérmico	14
2.10.2	Caso cuspidal adiabático	15
2.11	Ejemplo mínimo de uso	15
2.12	Limitaciones y consideraciones de implementación	17
2.13	Referencias	17

Capítulo 2

Spikes de materia oscura alrededor de agujeros negros supermasivos: implementación computacional y documentación del código

2.1 Objetivo y alcance

La librería implementa la parte del cálculo de Gondolo y Silk (1999) que conduce desde una función de distribución final $f'(E', L')$ hasta un perfil de densidad $\rho'(r)$. El dominio físico adoptado es la región interior en la que el agujero negro domina el potencial gravitatorio y este puede tratarse como kepleriano.

El código cubre dos modelos iniciales:

1. **Núcleo finito isotérmico.** Cerca del agujero negro se utiliza la aproximación constante de la función de distribución isotérmica.
2. **Cúspide de ley de potencia.** Se considera

$$\rho_i(r) = \rho_0 \left(\frac{r}{r_0} \right)^{-\gamma}, \quad 0 < \gamma < 2,$$

y se construye $f'(E', L')$ mediante la conservación del momento angular y de la acción radial.

El resultado calculado es $\rho'(r)$ antes de imponer una meseta de densidad producida por la autoaniquilación. Por tanto, la prescripción

$$\rho_{\text{sp}}(r) = \frac{\rho'(r)\rho_{\text{core}}}{\rho'(r) + \rho_{\text{core}}}$$

no forma parte de la implementación actual.

2.2 Organización de la librería

La distribución sigue el esquema *src layout*:

```
src/  
'-- dm_spikes/  
    |-- __init__.py  
    |-- density.py
```

```
|-- isothermal.py
|-- adiabatic_cusp.py
'-- storage.py
```

La carpeta `src` delimita el código fuente, mientras que `dm_spikes` constituye el paquete importable. Tras instalar el proyecto en modo editable desde su directorio raíz mediante

```
python -m pip install -e .
```

el paquete se importa como `dm_spikes` y no como `src.dm_spikes`.

2.2.1 Responsabilidades de los módulos

Módulo	Responsabilidad
<code>density.py</code>	Cinemática orbital, límites de captura e integración común de la densidad final.
<code>isothermal.py</code>	Construcción de la función de distribución constante y cálculo del perfil isotérmico.
<code>adiabatic_cusp.py</code>	Modelo cuspidal de Gondolo–Silk, acciones radiales, mapeo adiabático y cálculo del perfil final para un valor de γ .
<code>storage.py</code>	Guardado, carga y listado de arreglos de resultados en archivos <code>.npz</code> .
<code>__init__.py</code>	Definición de la interfaz pública del paquete.

Cuadro 2.1: Separación de responsabilidades entre los módulos de la librería.

La dependencia interna es unidireccional: `isothermal.py` y `adiabatic_cusp.py` dependen de `density.py`, mientras que este último no conoce los modelos físicos particulares. Por su parte, `storage.py` es independiente del cálculo físico.

2.3 Dependencias

Dependencia	Elementos utilizados	Finalidad
<code>numpy</code>	<code>array</code> , <code>sqrt</code> , <code>maximum</code> , logaritmos y exponenciales	Operaciones numéricas, protección de radicales y manejo de arreglos.
<code>scipy.integrate</code>	<code>quad</code>	Cuadratura adaptativa de las integrales sobre E' y sobre la variable transformada u .
<code>scipy.optimize</code>	<code>root_scalar</code>	Resolución numérica escalar de la ecuación de conservación de la acción radial.
<code>scipy.special</code>	<code>beta</code> , <code>gamma</code>	Cálculo del coeficiente $b(\gamma)$ y evaluación numéricamente estable del cociente de funciones gamma.
<code>pathlib</code>	<code>Path</code>	Construcción portable de rutas y búsqueda de archivos de resultados.
<code>datetime</code>	<code>datetime.now</code>	Incorporación de la fecha y la hora en el nombre de cada archivo guardado.

Cuadro 2.2: Dependencias utilizadas por la librería y su finalidad.

Matplotlib no constituye una dependencia interna de la librería, pues se utiliza únicamente en el notebook para visualizar los arreglos devueltos. Asimismo, el bucle sobre varios valores de γ pertenece al nivel del notebook y no a la función `compute_cusp_profile`.

2.4 Convenciones físicas y unidades

La implementación no utiliza una biblioteca de unidades. Por tanto, las magnitudes deben interpretarse de acuerdo con la Tabla 2.3.

Magnitud	Símbolo	Unidad esperada
Radio	r, r_0, D	pc
Masa del agujero negro	M	$M_\odot$
Velocidad	v, σ_v, c	km s^{-1}
Energía específica	E, E'	$(\text{km s}^{-1})^2$
Momento angular específico	L, L'	pc km s^{-1}
Densidad	ρ	$M_\odot \text{ pc}^{-3}$
Función de distribución	f	$M_\odot \text{ pc}^{-3} (\text{km s}^{-1})^{-3}$
Acción radial específica	I	pc km s^{-1}

Cuadro 2.3: Convención de unidades empleada por el código.

Las constantes numéricas utilizadas son

$$G = 4.30091 \times 10^{-3} \text{ pc } (\text{km s}^{-1})^2 M_\odot^{-1}, \quad (2.1)$$

$$c = 299792.458 \text{ km s}^{-1}. \quad (2.2)$$

En el potencial final, las órbitas ligadas satisfacen $E' < 0$. En cambio, en el modelo cuspidal inicial de Gondolo–Silk, la variable E utilizada por la implementación es positiva. Esta diferencia de convención está incorporada en las validaciones del módulo `adiabatic_cusp.py`.

2.5 Módulo `density.py`: integración

2.5.1 Modelo de Gondolo-Silk

La densidad final se calcula mediante

$$\rho'(r) = \int_{E'_m}^0 dE' \int_{L'_c}^{L'_m} dL' \frac{4\pi L'}{r^2 v_r} f'(E', L'), \quad (2.3)$$

donde

$$v_r = \left[2 \left(E' + \frac{GM}{r} - \frac{L'^2}{2r^2} \right) \right]^{1/2}, \quad (2.4)$$

$$E'_m = -\frac{GM}{r} \left(1 - \frac{4R_S}{r} \right), \quad (2.5)$$

$$L'_c = 2cR_S, \quad (2.6)$$

$$L'_m = \left[2r^2 \left(E' + \frac{GM}{r} \right) \right]^{1/2}, \quad (2.7)$$

$$R_S = \frac{2GM}{c^2}. \quad (2.8)$$

Se omiten las órbitas no ligadas, correspondientes a $E' > 0$. El límite L'_c excluye las partículas capturadas por el agujero negro. Asimismo, el código establece

$$\rho'(r) = 0, \quad r \leq 4R_S. \quad (2.9)$$

2.5.2 Eliminación de la singularidad en el extremo angular

La integral original contiene una singularidad integrable cuando $L' \rightarrow L'_m$, pues en dicho límite $v_r \rightarrow 0$. A partir de la definición de L'_m se obtiene

$$v_r = \frac{1}{r} \sqrt{L_m'^2 - L'^2}. \quad (2.10)$$

Por consiguiente, el integrando angular contiene el factor

$$\frac{L'}{\sqrt{L_m'^2 - L'^2}}, \quad (2.11)$$

que diverge en el extremo superior de integración. Aunque esta singularidad es integrable, entregarla directamente al integrador adaptativo puede reducir la estabilidad y la precisión numéricas.

Para regularizar el integrando se introduce el cambio de variable

$$u = \sqrt{L_m'^2 - L'^2}. \quad (2.12)$$

Al diferenciar la relación anterior se obtiene

$$L' dL' = -u du. \quad (2.13)$$

Los nuevos límites de integración son

$$L' = L'_c \implies u = u_{\max} = \sqrt{L_m'^2 - L_c'^2}, \quad (2.14)$$

$$L' = L'_m \implies u = 0. \quad (2.15)$$

Por tanto, la integral angular se transforma como

$$\begin{aligned} & \int_{L'_c}^{L'_m} dL' \frac{4\pi L'}{r^2 v_r} f'(E', L') \\ &= \frac{4\pi}{r} \int_0^{u_{\max}} du f'\left(E', \sqrt{L_m'^2 - u^2}\right). \end{aligned} \quad (2.16)$$

Esta es la forma que evalúa realmente la función `rho_prime_at_r`. La transformación elimina explícitamente el denominador que se anula en $L' = L'_m$ y convierte el extremo singular en un integrando regular. Con ello se reduce la posibilidad de advertencias de convergencia o pérdidas de precisión en `quad`.

2.5.3 Estabilidad numérica

La implementación incorpora las siguientes medidas de protección:

- Los radicales se evalúan mediante expresiones de la forma `sqrt(maximum(x, 0.0))`. Esto evita la aparición de valores NaN cuando los errores de redondeo producen valores ligeramente negativos en una frontera física.

- Si

$$L'_m{}^2 \leq L'_c{}^2,$$

el intervalo angular físicamente permitido está vacío. En ese caso, la contribución se fija en cero antes de calcular $u_{\max}$.

- Se exige que $r > 0$. Si $r \leq 4R_S$, la función devuelve cero sin intentar realizar la cuadratura.
- La misma configuración de `epsabs`, `epsrel` y `limit` se transmite a las cuadraturas interior y exterior. Los valores predeterminados son

$$\text{epsabs}=0.0, \quad \text{epsrel}=1\text{e-}6, \quad \text{limit}=100.$$

- El estimador de error devuelto por `quad` no se almacena. La API entrega únicamente el valor calculado de la integral. No obstante, las advertencias emitidas por SciPy siguen siendo relevantes para diagnosticar problemas de convergencia.

2.5.4 Referencia de funciones

`schwarzschild_radius`

Calcula el radio de Schwarzschild,

$$R_S = \frac{2GM}{c^2}. \quad (2.17)$$

Recibe la masa M en masas solares y devuelve el radio en pársecs. Los argumentos opcionales permiten sustituir los valores de G y c .

`energy_lower_bound`

Calcula $E'_m(r)$, el límite inferior de la integral sobre la energía. Para obtener R_S , llama internamente a `schwarzschild_radius`. Si $r > 4R_S$, el límite es negativo y la cuadratura exterior se realiza desde E'_m hasta cero.

`angular_momentum_capture`

Devuelve

$$L'_c = 2cR_S, \quad (2.18)$$

que representa el umbral de captura por el agujero negro. Esta cantidad no depende de r ni de E' .

`angular_momentum_max`

Calcula $L'_m(E', r, M)$. La llamada a `np.maximum` evita evaluar la raíz cuadrada de un valor negativo debido a errores de redondeo en los límites del dominio físico.

`radial_velocity`

Evalúa directamente $v_r(E', L', r, M)$. Esta función es útil para comprobaciones independientes y para representar la ecuación física. Sin embargo, la integral de densidad no la llama directamente, pues utiliza la forma regularizada de la Ecuación (2.16).

`rho_prime_at_r`

Constituye el núcleo numérico del módulo. Recibe una función invocable de la forma

$$\text{f_prime}(\text{Ep}, \text{Lp}),$$

un radio r y la masa M del agujero negro. La función construye los límites físicos, realiza la cuadratura anidada y devuelve un escalar correspondiente a $\rho'(r)$. Las excepciones producidas durante la evaluación de la función de distribución se propagan al código que realiza la llamada.

rho_prime_profile

Aplica **rho_prime_at_r** secuencialmente a cada elemento de **r_array** y reúne los resultados en un arreglo de NumPy. Esta operación no constituye una vectorización interna de **quad** ni ejecuta los radios en paralelo. La implementación utiliza una comprensión de lista que conserva el orden de la malla de entrada.

Los argumentos adicionales proporcionados mediante ****kwargs** se reenvían a **rho_prime_at_r**.

2.6 Módulo isothermal.py: núcleo finito

2.6.1 Modelo implementado

Para una esfera isotérmica, la función de distribución puede escribirse como

$$f(E, L) = \rho_0 (2\pi\sigma_v^2)^{-3/2} \exp\left(-\frac{E}{\sigma_v^2}\right). \quad (2.19)$$

En la aproximación utilizada cerca del agujero negro, $E \ll \sigma_v^2$, se toma

$$f'(E', L') \simeq f_0 = \rho_0 (2\pi\sigma_v^2)^{-3/2}. \quad (2.20)$$

La constancia de f_0 hace innecesario resolver explícitamente el mapeo de acciones para cada punto del espacio de fases. Esto no significa que el caso isotérmico sea ajeno a la hipótesis de crecimiento adiabático. En esta aproximación, la función de distribución es constante y, por tanto, su valor no cambia al componerla con el mapeo entre las variables iniciales y finales.

Como referencia analítica, la integral de densidad produce

$$\rho'_{\text{iso}}(r) = \frac{4\rho_0}{3\sqrt{\pi}} \left(\frac{GM}{r\sigma_v^2}\right)^{3/2} \left(1 - \frac{4R_S}{r}\right)^{3/2}. \quad (2.21)$$

Esta expresión es válida en la región interior considerada. La librería no emplea directamente esta fórmula para generar el perfil, sino que evalúa la integral común sobre el espacio de fases. En consecuencia, la Ecuación (2.21) puede utilizarse como una prueba independiente de la integración numérica.

2.6.2 Referencia de funciones

make_isothermal_distribution

Recibe ρ_0 y σ_v , calcula la constante

$$f_0 = \rho_0 (2\pi\sigma_v^2)^{-3/2}, \quad (2.22)$$

y devuelve una clausura con la interfaz

$$\mathbf{f_prime}(E_p, L_p).$$

Los argumentos E' y L' se aceptan para mantener la compatibilidad con la interfaz requerida por **rho_prime_profile**, pero se ignoran intencionalmente porque la función de distribución es constante.

La implementación actual no verifica que $\rho_0 > 0$ ni que $\sigma_v > 0$. Estas condiciones físicas deben comprobarse antes de llamar a la función.

`compute_isothermal_profile`

Construye la función de distribución constante mediante `make_isothermal_distribution` y, posteriormente, llama a `rho_prime_profile` para calcular el perfil de densidad sobre la malla radial.

Sus argumentos principales son el arreglo de radios, la masa M del agujero negro, la densidad de normalización ρ_0 y la dispersión de velocidades σ_v .

Los argumentos adicionales proporcionados mediante

`**integration_kwargs`

se transmiten a la rutina de integración común. Esto permite modificar, entre otros parámetros, las tolerancias numéricas `epsabs` y `epsrel`, así como el límite de subdivisiones `limit`.

2.7 Módulo `adiabatic_cusp.py`: cúspide adiabática

2.7.1 Perfil inicial y función de distribución

El perfil de densidad inicial está dado por

$$\rho_i(r) = \rho_0 \left(\frac{r}{r_0} \right)^{-\gamma}, \quad 0 < \gamma < 2. \quad (2.23)$$

Para este modelo se definen las cantidades

$$\phi_0 = \frac{4\pi G r_0^2 \rho_0}{(3 - \gamma)(2 - \gamma)}, \quad (2.24)$$

$$\beta = \frac{6 - \gamma}{2(2 - \gamma)}, \quad (2.25)$$

$$p = \frac{4 - \gamma}{2(2 - \gamma)}, \quad (2.26)$$

$$\lambda = \left(\frac{2}{4 - \gamma} \right)^{1/(2-\gamma)} \left(\frac{2 - \gamma}{4 - \gamma} \right)^{1/2}, \quad (2.27)$$

$$b = \frac{\pi(2 - \gamma)}{B(1/(2 - \gamma), 3/2)}. \quad (2.28)$$

La función de distribución inicial es

$$f_i(E, L) = \frac{\rho_0}{(2\pi\phi_0)^{3/2}} \frac{\Gamma(\beta)}{\Gamma(\beta - 3/2)} \left(\frac{\phi_0}{E} \right)^\beta. \quad (2.29)$$

Aunque la firma de la función conserva el argumento L , la expresión implementada es isotrópica y, por tanto, no depende explícitamente de este argumento.

2.7.2 Evaluación logarítmica de la función de distribución

La evaluación directa de $\Gamma(\beta)$, $\Gamma(\beta - 3/2)$ y $(\phi_0/E)^\beta$ puede producir valores intermedios extremadamente grandes o pequeños. Este problema adquiere particular importancia cuando γ se aproxima a 2, pues los exponentes β y p aumentan rápidamente.

Para mejorar la estabilidad numérica, el código calcula primero el logaritmo de la función de distribución:

$$\begin{aligned} \log f_i &= \log \rho_0 - \frac{3}{2} \log(2\pi\phi_0) + \log \Gamma(\beta) - \log \Gamma(\beta - 3/2) \\ &\quad + \beta (\log \phi_0 - \log E). \end{aligned} \quad (2.30)$$

Los términos $\log \Gamma(\beta)$ y $\log \Gamma(\beta - 3/2)$ se evalúan mediante `gammaLn`. Finalmente, la función retorna

$$f_i = \exp(\log f_i). \quad (2.31)$$

Esta reorganización evita calcular directamente funciones gamma potencialmente muy grandes y reduce la pérdida de rango numérico asociada a productos, potencias y cocientes. Sin embargo, no evita un posible desbordamiento final si el propio valor de f_i excede el rango representable en doble precisión.

2.7.3 Acciones radiales y mapeo adiabático

La acción radial inicial aproximada está dada por

$$I_i(E, L) = \frac{2\pi}{b} \left[-\frac{L}{\lambda} + \sqrt{2r_0^2\phi_0} \left(\frac{E}{\phi_0} \right)^p \right]. \quad (2.32)$$

Esta es la aproximación propuesta por Gondolo y Silk, cuyo error se reporta como inferior al 8% sobre el espacio de fases considerado [4].

En el potencial kepleriano final se utiliza la expresión analítica

$$I_f(E', L') = 2\pi \left[-L' + \frac{GM}{\sqrt{-2E'}} \right], \quad E' < 0. \quad (2.33)$$

La invariancia adiabática impone la conservación del momento angular y de la acción radial:

$$L' = L, \quad I_i(E, L') = I_f(E', L'). \quad (2.34)$$

En consecuencia, la función de distribución final se obtiene mediante

$$f'(E', L') = f_i(E, L'), \quad I_i(E, L') = I_f(E', L'). \quad (2.35)$$

Para cada par (E', L') , el código resuelve numéricamente la ecuación de conservación de la acción para recuperar la energía inicial positiva E . Posteriormente, evalúa $f_i(E, L')$.

2.7.4 Solución en la variable $\log E$

En lugar de aplicar el buscador de raíces directamente sobre E , se introduce la variable

$$x = \log E, \quad E = e^x. \quad (2.36)$$

De esta manera, la ecuación de conservación de la acción se transforma en

$$g(x) = I_i(e^x, L') - I_f(E', L') = 0. \quad (2.37)$$

La raíz se busca en el intervalo

$$x \in [\log E_{\min}, \log E_{\max}]. \quad (2.38)$$

Esta transformación presenta dos ventajas principales:

1. garantiza que $E > 0$ durante todas las iteraciones, sin necesidad de imponer restricciones adicionales al algoritmo;
2. permite representar de manera más equilibrada un intervalo energético que puede abarcar veinticuatro órdenes de magnitud.

Antes de llamar a `root_scalar`, la implementación verifica que:

- $E_{\min} > 0$ y $E_{\max} > 0$;
- $E_{\max} > E_{\min}$;
- los residuos evaluados en ambos extremos sean finitos;
- los residuos encierren un cambio de signo.

Las tolerancias `xtol` y `rtol` se aplican a la raíz en la coordenada logarítmica x , no directamente a la energía E .

En la rutina de alto nivel se utiliza el intervalo

$$(E_{\min}, E_{\max}) = (10^{-12}\phi_0, 10^{12}\phi_0). \quad (2.39)$$

La amplitud de este intervalo aumenta la posibilidad de encerrar la raíz para las órbitas evaluadas. La transformación logarítmica permite explorar dicho intervalo sin trabajar directamente con escalas energéticas separadas por un factor de 10^{24} .

2.7.5 Referencia de funciones auxiliares

`gs_phi0`

Calcula la escala de potencial

$$\phi_0 = \frac{4\pi G r_0^2 \rho_0}{(3 - \gamma)(2 - \gamma)}. \quad (2.40)$$

La función verifica que $0 < \gamma < 2$, $\rho_0 > 0$ y $r_0 > 0$. Devuelve una energía específica expresada en $(\text{km s}^{-1})^2$.

`gs_beta_exponent`

Calcula y devuelve el exponente

$$\beta = \frac{6 - \gamma}{2(2 - \gamma)}. \quad (2.41)$$

La función verifica que γ pertenezca al intervalo $(0, 2)$.

`gs_action_exponent`

Calcula el exponente

$$p = \frac{4 - \gamma}{2(2 - \gamma)} \quad (2.42)$$

que aparece en la expresión aproximada de la acción radial inicial.

gs_lambda

Calcula el coeficiente adimensional $\lambda(\gamma)$ empleado en la acción radial inicial. La función verifica que $0 < \gamma < 2$.

gs_b

Calcula el coeficiente

$$b = \frac{\pi(2 - \gamma)}{B(1/(2 - \gamma), 3/2)} \quad (2.43)$$

mediante `scipy.special.beta`. El uso de esta función especial evita implementar manualmente la función beta en términos de cocientes de funciones gamma.

initial_cusp_distribution

Evalúa la función de distribución inicial de la Ecuación (2.29) mediante la representación logarítmica de la Ecuación (2.30).

La función requiere

$$E > 0, \quad \rho_0 > 0, \quad r_0 > 0, \quad 0 < \gamma < 2. \quad (2.44)$$

También verifica que $\phi_0 > 0$ y $\beta > 3/2$. El argumento L se conserva para mantener la compatibilidad con la interfaz de las funciones de distribución en el espacio de fases, aunque la expresión implementada no depende de él.

initial_cusp_radial_action

Evalúa la acción radial inicial aproximada de la Ecuación (2.32). Requiere $E > 0$, $L \geq 0$ y $0 < \gamma < 2$.

radial_action_final_kepler

Evalúa la acción radial kepleriana de la Ecuación (2.33). Si $E' \geq 0$, devuelve `np.nan`, pues la expresión implementada solo se considera válida para órbitas ligadas.

solve_initial_energy_from_action

Resuelve numéricamente la ecuación

$$I_i(E, L') = I_f(E', L') \quad (2.45)$$

para obtener la energía inicial E .

La acción inicial, la acción final y sus argumentos adicionales se reciben como parámetros. De este modo, la función conserva cierto grado de generalidad sin introducir clases ni una jerarquía de modelos.

El argumento `E_bracket` puede ser:

- una tupla de la forma `(E_min, E_max)`;
- una función que reciba `(Ep, Lp, I_target)` y devuelva los límites energéticos.

La función resuelve la ecuación en $\log E$ y devuelve E después de exponenciar la raíz obtenida. Lanza una excepción si el intervalo es inválido, los residuos no encierran un cambio de signo o el algoritmo no converge.

make_f_prime_gs_cusp

Construye una clausura con la interfaz

$$f_prime_adiabatic(Ep, Lp).$$

Los parámetros γ , ρ_0 , r_0 y M quedan fijados dentro de la clausura. Para $E' \geq 0$, la función devuelve cero. Para $E' < 0$, resuelve la energía inicial mediante la conservación de la acción y evalúa $f_i(E, L')$.

Por defecto, cualquier error producido durante el mapeo se propaga al código que realizó la llamada. Si se establece

`return_zero_on_fail=True`,

las excepciones se convierten en una contribución nula. Esta opción puede ser útil durante una exploración preliminar, pero puede ocultar fallos de convergencia y no debe sustituir el diagnóstico del intervalo de acotación.

`compute_cusp_profile`

Constituye la rutina de alto nivel para calcular el perfil cuspidal final correspondiente a un valor de γ .

La normalización utilizada es

$$\rho_D = 0.0062 \left(1 - \frac{\gamma}{3}\right) M_\odot \text{pc}^{-3}, \quad r_0 = D, \quad (2.46)$$

con

$$D = 8.5 \text{ kpc} \quad (2.47)$$

por defecto, siguiendo la normalización del ejemplo galáctico de Gondolo y Silk.

La función construye el intervalo de energía de la Ecuación (2.39), genera la función de distribución final $f'(E', L')$ y llama a `rho_prime_profile` para realizar la integración sobre la malla radial.

Los parámetros numéricos predeterminados son

$$\text{epsrel} = 10^{-5}, \quad \text{limit} = 100. \quad (2.48)$$

El parámetro `epsabs` no se proporciona explícitamente y, por tanto, conserva el valor predeterminado

$$\text{epsabs} = 0.0 \quad (2.49)$$

definido en la rutina de integración común.

2.8 Módulo `storage.py`

`save_density_profiles`

Guarda los perfiles isotérmicos y cuspidales en un único contenedor `.npz`. Aunque ambos casos se almacenan en el mismo archivo, sus resultados permanecen separados mediante las siguientes claves:

`r_iso, rho_iso, gamma_values, r_cusp, rho_cusp_grid.`

La carpeta predeterminada es `results`. Si esta carpeta no existe, la función la crea antes de guardar los resultados.

El nombre del archivo adopta la forma

`perfiles_densidad_<etiqueta>_AAAAMDD_HHMMSS.npz.`

La marca temporal contiene el año, el mes, el día, la hora, los minutos y los segundos correspondientes al momento del guardado. Su inclusión reduce el riesgo de sobrescribir resultados obtenidos previamente.

Antes de escribir el archivo, la función verifica explícitamente que la ruta generada no exista. Si ya existe, lanza la excepción

`FileExistsError`.

Como la resolución de la marca temporal es de un segundo, dos operaciones de guardado realizadas con la misma etiqueta dentro del mismo segundo pueden generar exactamente el mismo nombre. En ese caso, la segunda operación produce la excepción mencionada y no sobrescribe el archivo existente.

El almacenamiento se realiza mediante `np.savez`. El formato `.npz` es un contenedor ZIP formado por uno o más archivos `.npy`. En este caso, los arreglos se almacenan sin compresión y conservan sus formas y tipos de datos de NumPy.

La función devuelve un objeto `Path` correspondiente a la ruta del archivo generado.

`load_density_profiles`

Abre un archivo `.npz` mediante `np.load` y extrae los arreglos asociados a las cinco claves esperadas. El resultado se devuelve como un diccionario ordinario de Python con la estructura

```
{
    "r_iso": r_iso,
    "rho_iso": rho_iso,
    "gamma_values": gamma_values,
    "r_cusp": r_cusp,
    "rho_cusp_grid": rho_cusp_grid,
}
```

La función supone que el archivo posee el esquema establecido por `save_density_profiles`. Si alguna de las claves esperadas no está presente, NumPy genera la excepción correspondiente.

La implementación tampoco comprueba explícitamente la compatibilidad entre las dimensiones de los arreglos. Por ejemplo, corresponde al usuario verificar que la primera dimensión de `rho_cusp_grid` coincida con la longitud de `gamma_values`.

`list_density_profiles`

Busca en la carpeta indicada todos los archivos cuyos nombres coincidan con el patrón

`perfiles_densidad_*.npz`.

Las rutas encontradas se devuelven como una lista ordenada lexicográficamente. Debido a que los nombres contienen una marca temporal con el formato `AAAAMDD_HHMMSS`, este orden lexicográfico coincide normalmente con el orden cronológico de creación para archivos que emplean la misma convención de nombres.

Si la carpeta especificada no existe o no contiene archivos compatibles con el patrón, la función devuelve una lista vacía.

2.9 Módulo `__init__.py`

El archivo `__init__.py` define la interfaz pública del paquete y reexporta las operaciones de uso más frecuente. Esto permite importar dichas funciones directamente desde `dm_spikes`, sin necesidad de indicar el módulo interno en el que están implementadas.

Las funciones reexportadas son:

- `schwarzschild_radius`;
- `rho_prime_at_r` y `rho_prime_profile`;
- `make_isothermal_distribution` y `compute_isothermal_profile`;
- `compute_cusp_profile`;
- `save_density_profiles`, `load_density_profiles` y `list_density_profiles`.

Por ejemplo, las funciones principales pueden importarse mediante

```
from dm_spikes import (
    compute_isothermal_profile,
    compute_cusp_profile,
    save_density_profiles,
)
```

Esta forma es equivalente, para las funciones reexportadas, a importarlas directamente desde sus módulos de implementación. Sin embargo, el uso de la interfaz pública reduce el acoplamiento entre el notebook y la organización interna de la librería.

Las funciones auxiliares específicas del modelo de Gondolo–Silk no se reexportan desde el nivel principal del paquete. Cuando son necesarias para una prueba numérica, una verificación o un desarrollo teórico, deben importarse directamente desde `adiabatic_cusp.py`. Por ejemplo,

```
from dm_spikes.adiabatic_cusp import (
    gs_phi0,
    initial_cusp_distribution,
    radial_action_final_kepler,
)
```

La variable `__all__` contiene explícitamente los nombres que constituyen la interfaz pública:

```
__all__ = [
    "schwarzschild_radius",
    "rho_prime_at_r",
    "rho_prime_profile",
    "make_isothermal_distribution",
    "compute_isothermal_profile",
    "compute_cusp_profile",
    "save_density_profiles",
    "load_density_profiles",
    "list_density_profiles",
]
```

Esta lista determina los nombres importados cuando se utiliza la expresión

```
from dm_spikes import *
```

2.10 Flujos de cálculo

2.10.1 Caso isotérmico

El cálculo del perfil isotérmico sigue la secuencia:

1. El usuario proporciona la masa M del agujero negro, la densidad ρ_0 , la dispersión de velocidades σ_v y una malla radial.
2. La función `make_isothermal_distribution` calcula

$$f_0 = \rho_0 (2\pi\sigma_v^2)^{-3/2}$$

y construye una clausura que representa la función de distribución constante.

3. La función `rho_prime_profile` recorre secuencialmente la malla radial.
4. Para cada radio, `rho_prime_at_r` determina los límites físicos y realiza las cuadraturas adaptativas sobre E' y sobre la variable transformada u .

5. Los valores calculados se reúnen en el arreglo `rho_iso`, conservando el mismo orden que la malla radial de entrada.

De forma esquemática, el flujo puede representarse como

$$(M, \rho_0, \sigma_v, \{r_j\}) \longrightarrow f_0 \longrightarrow \rho'_{\text{iso}}(r_j). \quad (2.50)$$

2.10.2 Caso cuspidal adiabático

El cálculo del perfil cuspidal adiabático sigue la secuencia:

1. El usuario proporciona la pendiente inicial γ , la masa M del agujero negro y una malla radial.
2. La función `compute_cusp_profile` fija ρ_0 , r_0 y ϕ_0 de acuerdo con la normalización empleada en el ejemplo galáctico.
3. La función `make_f_prime_gs_cusp` construye una clausura que representa la función de distribución final $f'(E', L')$.
4. La función `rho_prime_profile` recorre la malla radial y, para cada radio, `rho_prime_at_r` realiza las cuadraturas adaptativas sobre E' y u .
5. Cada evaluación de $f'(E', L')$ para $E' < 0$ llama a `solve_initial_energy_from_action`. Esta función resuelve

$$I_i(E, L') = I_f(E', L')$$

para obtener la energía inicial $E > 0$.

6. Una vez determinada E , se evalúa la función de distribución inicial $f_i(E, L')$, cuyo valor se asigna a $f'(E', L')$.
7. La integración produce el perfil final correspondiente a un único valor de γ .
8. El procedimiento completo se repite para cada elemento del arreglo `gamma_values`. Los perfiles resultantes se reúnen en `rho_cusp_grid`.

De forma esquemática, este flujo puede escribirse como

$$(\gamma, M, \{r_j\}) \longrightarrow f_i(E, L) \longrightarrow I_i(E, L') = I_f(E', L') \longrightarrow f'(E', L') \longrightarrow \rho'_{\text{cusp}}(r_j). \quad (2.51)$$

El flujo cuspidal es considerablemente más costoso que el isotérmico. En este caso, existe un solucionador de raíces dentro de la evaluación de una cuadratura adaptativa anidada. Este procedimiento se repite para cada radio y para cada valor de γ .

2.11 Ejemplo mínimo de uso

El siguiente ejemplo calcula el perfil isotérmico y los perfiles cuspidales correspondientes a varios valores de γ . Finalmente, reúne los resultados en un archivo `.npz` mediante una operación de guardado manual.

```
1 import numpy as np
2
3 from dm_spikes import (
4     compute_cusp_profile,
5     compute_isothermal_profile,
6     save_density_profiles,
7     schwarzschild_radius,
8 )
9
10 # Masa del agujero negro en masas solares
11 M = 2.6e6
12
13 # Radio de Schwarzschild en parsecs
```

```

14 R_S = schwarzschild_radius(M)
15
16 # Caso isotermico
17 r_iso = np.logspace(
18     np.log10(4.0001 * R_S),
19     np.log10(0.2),
20     200,
21 )
22
23 rho_iso = compute_isothermal_profile(
24     r_values=r_iso,
25     M=M,
26     rho0=1.0,
27     sigma_v=100.0,
28 )
29
30 # Caso cuspidal adiabatico
31 r_cusp = np.logspace(
32     np.log10(4.001 * R_S),
33     np.log10(0.2),
34     200,
35 )
36
37 gamma_values = np.array([
38     0.5,
39     1.0,
40     1.2,
41     1.4,
42     1.9,
43 ])
44
45 rho_cusp_grid = np.array([
46     compute_cusp_profile(
47         gamma=float(gamma),
48         r_values_adiabatic=r_cusp,
49         M=M,
50         epsrel=1e-5,
51         limit=100,
52     )
53     for gamma in gamma_values
54 ])
55
56 # Esta llamada debe ejecutarse manualmente cuando se desee guardar.
57 archivo = save_density_profiles(
58     r_iso=r_iso,
59     rho_iso=rho_iso,
60     gamma_values=gamma_values,
61     r_cusp=r_cusp,
62     rho_cusp_grid=rho_cusp_grid,
63     etiqueta="gs_gammas",
64 )

```

En este ejemplo, las dos mallas radiales comienzan ligeramente por encima de $4R_S$, donde el perfil deja de ser idénticamente nulo. El límite superior se fija en 0.2 pc, correspondiente a la región interior considerada en el cálculo.

El arreglo `rho_cusp_grid` tiene dimensiones

$$N_\gamma \times N_r, \quad (2.52)$$

donde N_γ es el número de valores incluidos en `gamma_values` y N_r es el número de puntos de la malla `r_cusp`. En este ejemplo,

$$N_\gamma = 5, \quad N_r = 200. \quad (2.53)$$

2.12 Limitaciones y consideraciones de implementación

La implementación actual presenta las siguientes limitaciones físicas, numéricas y operativas:

- El tratamiento es esencialmente newtoniano y utiliza un potencial final kepleriano, junto con el corte de captura prescrito por Gondolo y Silk. No realiza una integración relativista de geodésicas en las geometrías de Schwarzschild o Kerr.
- La integral considera únicamente órbitas ligadas, correspondientes a $E' < 0$. La contribución de las órbitas no ligadas, con $E' > 0$, se omite.
- El caso isotérmico emplea la aproximación constante de la función de distribución cerca del agujero negro:

$$f(E, L) \simeq \rho_0 (2\pi\sigma_v^2)^{-3/2}.$$

Por tanto, no se utiliza la dependencia exponencial completa de la distribución isotérmica.

- La acción radial inicial del modelo cuspidal se calcula mediante la aproximación analítica de Gondolo–Silk. No se evalúa directamente la integral radial exacta en el potencial inicial.
- La función `compute_cusp_profile` incorpora una normalización galáctica específica mediante D y ρ_D . Para estudiar un halo con una normalización diferente sería conveniente implementar una rutina de menor nivel que reciba explícitamente ρ_0 y r_0 .
- Las unidades físicas están documentadas, pero no se representan mediante objetos con unidades ni se verifican automáticamente. El usuario debe proporcionar todas las magnitudes en el sistema de unidades adoptado por la librería.
- Las llamadas a `quad` devuelven tanto el valor aproximado de la integral como una estimación del error absoluto. La implementación conserva únicamente el valor de la integral; las estimaciones de error no se almacenan junto con el perfil calculado.
- La función `rho_prime_profile` recorre secuencialmente la malla radial. No existe paralelización sobre los radios ni sobre los valores de γ . Esta decisión mantiene la implementación simple y reproducible, pero limita el rendimiento para mallas extensas.
- En el caso cuspidal, cada evaluación de la función de distribución final puede requerir la solución numérica de una ecuación de conservación de la acción. La presencia de un solucionador de raíces dentro de una cuadratura adaptativa anidada constituye el principal coste computacional.
- La opción `return_zero_on_fail=True` transforma cualquier excepción durante la construcción de $f'(E', L')$ en una contribución nula. Aunque esta opción puede facilitar exploraciones preliminares, también puede ocultar problemas de convergencia o intervalos de acotación inadecuados.
- El formato `.npz` conserva los arreglos y sus tipos de datos, pero el esquema actual no almacena metadatos físicos o numéricos como M , ρ_0 , σ_v , las tolerancias de integración, las unidades o la versión del código.

2.13 Referencias

- [1] J. Aschersleben, G. Bertone, D. Horns, E. Moulin, R. F. Peletier y M. Vecchi, “Gamma rays from dark matter spikes in EAGLE simulations”, *Journal of Cosmology and Astroparticle Physics*, núm. 09, art. 005 (2024).
doi:10.1088/1475-7516/2024/09/005.

- [2] G. Bertone, A. R. Zentner y J. Silk, “A new signature of dark matter annihilations: Gamma-rays from intermediate-mass black holes”, *Physical Review D*, vol. 72, art. 103517 (2005). doi:10.1103/PhysRevD.72.103517.
- [3] M. Cirelli, A. Strumia y J. Zupan, “Dark matter”, *SciPost Physics Reviews*, art. 1 (2026). doi:10.21468/SciPostPhysRev.1.
- [4] P. Gondolo y J. Silk, “Dark matter annihilation at the galactic center”, *Physical Review Letters*, vol. 83, pp. 1719–1722 (1999). doi:10.1103/PhysRevLett.83.1719.
- [5] L. Sadeghian, F. Ferrer y C. M. Will, “Dark matter distributions around massive black holes: A general relativistic analysis”, *Physical Review D*, vol. 88, art. 063522 (2013). doi:10.1103/PhysRevD.88.063522.
- [6] SciPy Developers, “`scipy.integrate.quad`”, documentación de SciPy.
<https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.quad.html>.
- [7] SciPy Developers, “`scipy.optimize.root_scalar`”, documentación de SciPy.
https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.root_scalar.html.
- [8] N. Speeney, A. Antonelli, V. Baibhav y E. Berti, “The impact of relativistic corrections on the detectability of dark-matter spikes with gravitational waves”, *Physical Review D*, vol. 106, art. 044027 (2022). doi:10.1103/PhysRevD.106.044027.
- [9] P. Ullio, H. Zhao y M. Kamionkowski, “A dark-matter spike at the Galactic Center?”, *Physical Review D*, vol. 64, art. 043504 (2001). doi:10.1103/PhysRevD.64.043504.